

Midterm

*Instructor: Matthew Green**In class*

Name: _____

This midterm should be completed individually in class. Do not use devices or collaborate with any other student. You are permitted to use a single (two-sided) sheet of notes if you prepared it yourself.

Problem 1: True, False or Unknown (25 points). Mark each question with T/F/U. You do not need to explain your work, but you can.

1. **T/F/U.** If a hash function H is collision-resistant, then a Merkle tree built using H is also collision-resistant.
2. **T/F/U.** In Bitcoin, every single `scriptPubKey` script must include a digital signature verification opcode.
3. **T/F/U.** Assume that the Bitcoin hash function behaves like a random oracle (random function). Consider a miner that has been unsuccessfully mining on a block header for 9.5 minutes. A new transaction arrives, and the miner wishes to include this transaction into the block. Assume that the mining hardware does not slow down if the miner changes the block hash it is mining on. The miner should *not* include the new transaction, as the change will waste previous effort.
4. **T/F/U.** Transactions in the Ethereum “mempool” are validated by executing them against the most recent block in the consensus blockchain, before they are included into a block by a miner/validator.

5. **T/F/U.** The selfish mining attack requires the attacker to hold 51% of the network hashpower.
6. **T/F/U.** Ethereum smart contracts can hold secret variables such as digital signing keys.
7. **T/F/U.** Miner Extractable Value (MEV) can exploit trading transactions (e.g., on an AMM) by placing chosen transactions before and *after* a target transaction.
8. **T/F/U.** An optimistic rollup requires some form of *data availability* so that volunteers (validators) can see all transactions executed by the rollup.
9. **T/F/U.** A privacy system such as Tornado Cash (discussed in class) must keep track of all “nullifiers” added to the system.
10. **T/F/U.** Ethereum initialization code (used in constructors) is not included in the hash of a contract bytecode returned by the opcode `EXTCODEHASH`.

Problem 2: Medium answers (40 points)

1. Computing smart contract addresses.

Compare and contrast two options for creating smart contract addresses in Ethereum. Explain the pros and cons of each approach, and which is better.

- (a) Let's say we compute a new Ethereum contract address as `contract_address` $\leftarrow \text{SHA3}(\text{sender_address} \parallel \text{code})$ at creation time, where `sender_address` is the wallet address of the user who issues the creation transaction, `code` is the contract EVM bytecode and $\parallel$ means concatenation of strings.
- (b) Let's say we compute a new Ethereum contract address as `contract_address` $\leftarrow \text{SHA3}(\text{sender_address} \parallel \text{nonce})$ at creation time, where `nonce` is a nonce enforced by the Ethereum network. **Note:** Explain what conditions you'd need the network to enforce on `nonce`.

2. **MEV.** Imagine that you wish to stop Miner Extractable Value (MEV) behavior on a network such as Ethereum. MEV involves miners/validators re-ordering transactions and adding new transactions to Ethereum blocks in order to extract value from e.g., trading smart contracts.

One proposed improvement to Ethereum-like networks is to require each transaction sender to perform their transactions in two steps. In a first *commit* step, the sender will send a hash of their transaction to the network, where it must be included in a block. In a second step, the sender will send the actual transaction to the network. The transaction can only be included in a block if its hash was included in a previous block. Explain how this might or might not stop MEV attacks.

3. **Tornado Cash.** In the privacy system we discussed in class (modeling Tornado Cash) we described two protocols.

A **Deposit** protocol deposited funds into a smart contract, along with a “note commitment” of the form $h = H(sk_1 || sk_2)$ where sk_1, sk_2 were two secret strings chosen by the depositor. The smart contract would record all note commitments it received in a list.

To reclaim the same amount of funds, the user would execute a **Withdraw** protocol that would send the smart contract a “nullifier” sk'_2 along with a zero-knowledge proof of what statement? (You can write the statement in English, you do not need to provide a program.) Explain why the protocol worked (from a privacy and financial soundness perspective.)

Problem 3: Solidity stuff (35 points)

You are developing a financial platform to support Web3 gaming. Your system has three contracts: `GameToken`, `GameTokenManager` and `GameTokenRegistry`. Each `GameToken` represents a token for a specific game, and there can be many of them. There is only one canonical instance of each of the Manager and Registry contracts. The requirements are as follows:

1. Game developers are expected to create a new `GameToken` instance themselves, and register it into the single instance of `GameTokenRegistry`.
2. All tokens should have the same total supply of 89457938388334834845739534 units.
3. When a new token is created and registered, the `GameTokenManager` contract should be the initial owner of all newly-created funds.
4. Each time some user calls the `transfer()` method of `GameToken`, the `GameTokenManager` contract must approve the transfer, or else it will abort.
5. Aside from blocking transactions, bugs in `GameTokenManager` should not put user-owned funds at any risk.

Your Solidity developers have produced the following code for `GameToken` and `GameTokenRegistry`. Does it satisfy these requirements? List any flaws or even potential concerns. (We are not giving code for `GameTokenManager`, but you can assume that any methods we reference below exist and are called properly.)

```
contract GameToken is IERC20 { // ERC20 interface
    // hardcoded address of the GameTokenManager contract
    address immutable _gameManager =
        0x74Cef96B8de1531a94613a6B1d53BE2BBEF974ff;

    ... // standard ERC20 variables such as totalSupply, balanceOf[]

    // Constructor
    constructor() {
        totalSupply = 89457938388334834845739534; // A fixed initial supply
        // All tokens initially owned by game manager
        balanceOf[_gameManager] = totalSupply;
    }

    function transfer(address recipient, uint256 amount)
        external
        returns (bool)
    {
        GameTokenManager manager = GameTokenManager(_gameManager);
```

```

        balanceOf[recipient] += amount;
        // Make sure the game manager (contract) approves this!
        require (manager.approveTransfer(msg.sender, amount, recipient) == true);
        balanceOf[msg.sender] -= amount;
        return true;
    }

    ... // other methods omitted
}

contract GameTokenRegistry {
    // hardcoded hash of GameToken contract bytecode
    bytes32 _knownTokenContractHash =
        0xc5d2460186f7233c927e7db2dcc703c0e500b653ca82273b7bfad8045d85a470;

    mapping(address => uint256) private _registeredContracts;
    ...

    // Allow external callers to add a new GameToken contract to the registry.
    // We must check that it's really a GameToken, using EXTCODEHASH.
    function registerNewToken(GameToken newToken) external {
        // We want to be sure this contract truly is a GameToken, so we'll
        // check its code hash
        bytes32 codeHash;
        assembly { codeHash := extcodehash(address(newToken)) }
        require (codeHash == _knownTokenContractHash);
        _registeredContracts[address(newToken)] = 1; // add to registry
    }
}

```

(this page left blank to allow space for answers)

(this page left blank to allow space for answers)